

OPERATING SYSTEM

PROJECT 2 – Page Tables and Traps

1. General rule:

- The project is done in groups: each group has a maximum of **3 students**
- **The same exercises will all be scored 0 for the entire practice (even though there are scores for other exercises and practice projects).**
- Môi trường lập trình: **Linux**

2. Submission:

Submit assignments directly on the course website (MOODLE), not accepting submissions via email or other forms.

Filename: **StudentID1_StudentID2_StudentID3.zip** (with StudentID1 < StudentID2 < StudentID3)

Ex: Your group has 2 students: 2312001, 2312002 and 2312003, the filename is:
2312001_2312002_2312003.zip

Include:

- **StudentID1_StudentID2_StudentID3_Report.pdf:** Writeups should be short and sweet. Do not spend too much effort or include your source code on your writeups. The purpose of the report is to give you an opportunity to clarify your solution, any problems with your work, and to add information that may be useful in grading. If you had specific problems or issues, approaches you tried that didn't work, or concepts that were not fully implemented, then an explanation in your report may help us to assign partial credit
- **Release:** File diff (diff patch, Ex: \$ git diff > <StudentID1_StudentID2_StudentID3>.patch)
- **Source:** Zip file of xv6 (the version is made clean)

Folder Structure Example:

```
page_table/  
StudentID1_StudentID2_StudentID3_Report.pdf  
StudentID1_StudentID2_StudentID3.patch  
xv6_page_table.zip
```

```
traps/  
  StudentID1_StudentID2_StudentID3_Report.pdf  
  StudentID1_StudentID2_StudentID3.patch  
  xv6_traps.zip
```

Lưu ý: Cần thực hiện đúng các yêu cầu trên, nếu không, bài làm sẽ không được chấm.

3. Demo Interviews

Your implementation is graded on completeness, correctness, programming style, thoroughness of testing, your solution, and code understanding.

When administering this course, we do our best to give a fair assessment to each individual based on each person's contribution to the project

4. Requirements

4.1 Page Tables

In this lab you will explore page tables and modify them to speed up certain system calls and to detect which pages have been accessed.

Before you start coding, read Chapter 3 of the [xv6 book](#), and related files:

- kernel/memlayout.h, which captures the layout of memory.
- kernel/vm.c, which contains most virtual memory (VM) code.
- kernel/kalloc.c, which contains code for allocating and freeing physical memory.

It may also help to consult the [RISC-V privileged architecture manual](#).

To start the lab, switch to the pgtbl branch:

```
$ git fetch  
$ git checkout pgtbl  
$ make clean
```

4.1.1 Speed up system calls

Some operating systems (e.g., Linux) speed up certain system calls by sharing data in a read-only region between userspace and the kernel. This eliminates the need for kernel crossings when performing these system calls. To help you learn how to insert mappings into a page table, your first task is to implement this optimization for the `getpid()` system call in `xv6`.

When each process is created, map one read-only page at `USYSCALL` (a virtual address defined in `memlayout.h`). At the start of this page, store a struct `usyscall` (also defined in `memlayout.h`), and initialize it to store the PID of the current process. For this lab, `ugetpid()` has been provided on the userspace side and will automatically use the `USYSCALL` mapping. You will receive full credit for this part of the lab if the `ugetpid` test case passes when running `pgtbltest`.

Some hints:

- Choose permission bits that allow userspace to only read the page.
- There are a few things that need to be done over the lifecycle of a new page. For inspiration, understand the trapframe handling in `kernel/proc.c`.

Which other `xv6` system call(s) could be made faster using this shared page? Explain how.

4.1.2 Print a page table

To help you visualize RISC-V page tables, and perhaps to aid future debugging, your second task is to write a function that prints the contents of a page table.

Define a function called `vmprint()`. It should take a `pagetable_t` argument, and print that pagetable in the format described below. Insert `if(p->pid==1) vmprint(p->pagetable)` in `exec.c` just before the return `argc`, to print the first process's page table. You receive full credit for this part of the lab if you pass the `pte printout` test of `make grade`.

Now when you start `xv6` it should print output like this, describing the page table of the first process at the point when it has just finished `exec()`ing `init`:

```
page table 0x0000000087f6b000
..0: pte 0x0000000021fd9c01 pa 0x0000000087f67000
.. ..0: pte 0x0000000021fd9801 pa 0x0000000087f66000
.. .. ..0: pte 0x0000000021fda01b pa 0x0000000087f68000
.. .. ..1: pte 0x0000000021fd9417 pa 0x0000000087f65000
.. .. ..2: pte 0x0000000021fd9007 pa 0x0000000087f64000
.. .. ..3: pte 0x0000000021fd8c17 pa 0x0000000087f63000
..255: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..511: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. ..509: pte 0x0000000021fdcc13 pa 0x0000000087f73000
```

```
.. .. .510: pte 0x0000000021fdd007 pa 0x0000000087f74000
.. .. .511: pte 0x0000000020001c0b pa 0x0000000080007000
init: starting sh
```

The first line displays the argument to `vmprint`. After that there is a line for each PTE, including PTEs that refer to page-table pages deeper in the tree. Each PTE line is indented by a number of "." that indicates its depth in the tree. Each PTE line shows the PTE index in its page-table page, the pte bits, and the physical address extracted from the PTE. Don't print PTEs that are not valid. In the above example, the top-level page-table page has mappings for entries 0 and 255. The next level down for entry 0 has only index 0 mapped, and the bottom-level for that index 0 has entries 0, 1, and 2 mapped.

Your code might emit different physical addresses than those shown above. The number of entries and the virtual addresses should be the same.

Some hints:

- You can put `vmprint()` in `kernel/vm.c`.
- Use the macros at the end of the file `kernel/riscv.h`.
- The function `freewalk` may be inspirational.
- Define the prototype for `vmprint` in `kernel/defs.h` so that you can call it from `exec.c`.
- Use `%p` in your `printf` calls to print out full 64-bit hex PTEs and addresses as shown in the example.

For every leaf page in the `vmprint` output, explain what it logically contains and what its permission bits are. Figure 3.4 in the `xv6` book might be helpful, although note that the figure might have a slightly different set of pages than the `init` process that's being inspected here.

4.1.3 Detect which pages have been accessed

Some garbage collectors (a form of automatic memory management) can benefit from information about which pages have been accessed (read or write). In this part of the lab, you will add a new feature to `xv6` that detects and reports this information to userspace by inspecting the access bits in the RISC-V page table. The RISC-V hardware page walker marks these bits in the PTE whenever it resolves a TLB miss.

Your job is to implement `pgaccess()`, a system call that reports which pages have been accessed. The system call takes three arguments. First, it takes the starting virtual address of the first user page to check. Second, it takes the number of pages to check. Finally, it takes a user address to a buffer to store the results into a bitmask (a datastructure that uses one bit per page and where the first page corresponds to the

least significant bit). You will receive full credit for this part of the lab if the pgaccess test case passes when running pgtbltest.

Some hints:

- Read pgaccess_test() in user/pgtbltest.c to see how pgaccess is used.
- Start by implementing sys_pgaccess() in kernel/sysproc.c.
- You'll need to parse arguments using argaddr() and argint().
- For the output bitmask, it's easier to store a temporary buffer in the kernel and copy it to the user (via copyout()) after filling it with the right bits.
- It's okay to set an upper limit on the number of pages that can be scanned.
- walk() in kernel/vm.c is very useful for finding the right PTEs.
- You'll need to define PTE_A, the access bit, in kernel/riscv.h. Consult the [RISC-V privileged architecture manual](#) to determine its value.
- Be sure to clear PTE_A after checking if it is set. Otherwise, it won't be possible to determine if the page was accessed since the last time pgaccess() was called (i.e., the bit will be set forever).
- vmprint() may come in handy to debug page tables.

4.2 Traps

This lab explores how system calls are implemented using traps. You will first do a warm-up exercises with stacks and then you will implement an example of user-level trap handling.

Before you start coding, read Chapter 4 of the xv6 book, and related source files:

- kernel/trampoline.S: the assembly involved in changing from user space to kernel space and back
- kernel/trap.c: code handling all interrupts

To start the lab, switch to the trap branch:

```
$ git fetch
$ git checkout traps
$ make clean
```

4.2.1 RISC-V assembly (easy)

It will be important to understand a bit of RISC-V assembly. There is a file user/call.c in your xv6 repo. `make fs.img` compiles it and also produces a readable assembly version of the program in user/call.asm.

Read the code in call.asm for the functions g, f, and main. The instruction manual for RISC-V is on the reference page. Answer the following questions in answers-traps.txt:

Which registers contain arguments to functions? For example, which register holds 13 in main's call to printf?

Where is the call to function f in the assembly code for main? Where is the call to g? (Hint: the compiler may inline functions.)

At what address is the function printf located?

What value is in the register ra just after the jalr to printf in main?

Run the following code.

```
unsigned int i = 0x00646c72;  
printf("H%x Wo%s", 57616, (char *) &i);
```

What is the output? [Here's an ASCII table](#) that maps bytes to characters.

The output depends on that fact that the RISC-V is little-endian. If the RISC-V were instead big-endian what would you set `i` to in order to yield the same output? Would you need to change `57616` to a different value?

[Here's a description of little- and big-endian](#) and [a more whimsical description](#).

In the following code, what is going to be printed after '`y=`'? (note: the answer is not a specific value.) Why does this happen?

```
printf("x=%d y=%d", 3);
```

4.2.2 Backtrace (moderate)

For debugging it is often useful to have a backtrace: a list of the function calls on the stack above the point at which the error occurred. To help with backtraces, the compiler generates machine code that maintains a stack frame on the stack corresponding to each function in the current call chain. Each stack frame consists of the return address and a "frame pointer" to the caller's stack frame. Register `s0` contains a pointer to the current stack frame (it actually points to the the address of the saved return address on the stack plus 8). Your backtrace should use the frame pointers to walk up the stack and print the saved return address in each stack frame.

Implement a `backtrace()` function in `kernel/printf.c`. Insert a call to this function in `sys_sleep`, and then run `bttest`, which calls `sys_sleep`. Your output should be a list of return

addresses with this form (but the numbers will likely be different):

```
backtrace:
0x0000000080002cda
0x0000000080002bb6
0x0000000080002898
```

After bttest exit qemu. In a terminal window: run `addr2line -e kernel/kernel` (or `riscv64-unknown-elf-addr2line -e kernel/kernel`) and cut-and-paste the addresses from your backtrace, like this:

```
$ addr2line -e kernel/kernel
0x0000000080002de2
0x0000000080002f4a
0x0000000080002bfc
Ctrl-D
```

You should see something like this:

```
kernel/sysproc.c:74
kernel/syscall.c:224
kernel/trap.c:85
```

Some hints:

- Add the prototype for your `backtrace()` to `kernel/defs.h` so that you can invoke `backtrace` in `sys_sleep`.
- The GCC compiler stores the frame pointer of the currently executing function in the register `s0`. In the section marked by `#ifndef __ASSEMBLER__ ... #endif`, add the following function to `kernel/riscv.h`:

```
static inline uint64
r_fp()
{
    uint64 x;
    asm volatile("mv %0, s0" : "=r" (x) );
    return x;
}
```

and call this function in `backtrace` to read the current frame pointer. `r_fp()` uses [in-line assembly](#) to read `s0`.

- These [lecture notes](#) have a picture of the layout of stack frames. Note that the return address lives at a fixed offset (-8) from the frame pointer of a stackframe, and that the saved frame pointer lives at fixed offset (-16) from the frame pointer.

- Your `backtrace()` will need a way to recognize that it has seen the last stack frame, and should stop. A useful fact is that the memory allocated for each kernel stack consists of a single page-aligned page, so that all the stack frames for a given stack are on the same page. You can use `PGROUNDDOWN(fp)` (see `kernel/riscv.h`) to identify the page that a frame pointer refers to.

Once your backtrace is working, call it from `panic` in `kernel/printf.c` so that you see the kernel's backtrace when it panics.

5. Grade

No.	Exercise	Grade
1	Speed up system calls	1.5
2	Print a page table	2
3	Detect which pages have been accessed	3
4	RISC-V assembly	1.5
5	Backtrace	2

6. Reference

- <https://pdos.csail.mit.edu/6.1810/2023/labs/pgtbl.html>